



ESCUELA POLITÉCNICA NACIONAL

Sistema de Gestión de Tutorías Académicas

v1.0

Carlos Caviedes

Beren Quing

Programación I

13 de Julio de 2026

2. Introducción

El presente sistema es una solución desarrollada en C para gestionar tutorías mediante persistencia de datos en memoria secundaria.

Su objetivo es automatizar la asignación académica.

El propósito de este manual técnico es documentar la arquitectura, las librerías empleadas y el manejo de estructuras para facilitar el mantenimiento y escalabilidad por parte de otros desarrolladores.

3. Descripción del Sistema

Objetivo del software: Proveer una plataforma de consola para el agendamiento y control de tutorías.

Problema que resuelve: Elimina el uso de papel y la desorganización en las tutorías, automatizando el cruce de horarios y validando las inscripciones de forma dinámica.

Funcionalidades principales: Autenticación dinámica, manejo de estado lógico de registros, generación algorítmica de estadísticas e integridad referencial entre módulos.

4. Tecnologías Utilizadas

Lenguaje de programación: C estándar.

Entorno de desarrollo: Zinjai.

Sistema operativo: Windows.

Herramientas o librerías empleadas: <stdio.h>, <stdlib.h> (manejo de memoria y consola), <string.h> (manipulación de cadenas y validaciones), <ctype.h> (análisis de caracteres).

5. Arquitectura y Estructura

Diagrama simple de módulos: main.c, tutorias.h/c, estudiantes. h/c y docentes.h/c.

Descripción de los principales archivos o carpetas: El proyecto se divide en módulos que separan las estructuras (structs) y las funciones en sus respectivas cabeceras (.h) y cuerpos (.c).

Funciones principales del sistema: Uso de arreglos de estructuras (listaTutorias, listaDocentes) y funciones de serialización (fopen, fprintf, fscanf) para la persistencia en archivos planos (.txt).

6. Instalación y Ejecución

Requisitos mínimos: Compilador instalado.

Pasos para compilar: Importar todos los archivos fuente al IDE Zinjai y ejecutar el comando de compilación (F9), asegurando que las cabeceras estén en el mismo directorio.

Pasos para ejecutar la aplicación: Abrir el ejecutable compilado. El sistema creará automáticamente los archivos .txt si es la primera ejecución.

7. Validaciones y Manejo de Errores

Validaciones implementadas: Control mediante ciclos do-while, uso de strcmp para evitar duplicidad de códigos y validaciones de tipo de dato con isdigit e isalpha.

Mensajes de error más importantes: Errores de violaciones de integridad referencial "No existen docentes en el sistema" e intentos de salto de línea residuales (solucionados limpiando el buffer con while(getchar() != '\n')).

Soluciones básicas a problemas frecuentes: Si el archivo se corrompe por cierres forzados, se recomienda borrar los archivos .txt para reiniciar los arreglos a cero.

8. Mantenimiento y Mejoras Futuras

Cómo agregar nuevas funcionalidades: Para incluir un módulo nuevo, por ejemplo "Aulas", se debe crear un nuevo par aulas.h y aulas.c, definir su struct y agregarlo a las llamadas de guardado y carga en el main.c.

Recomendaciones para futuras versiones: Transicionar de memoria basada en arreglos estáticos a memoria dinámica (malloc), e implementar un algoritmo de encriptación básico para las contraseñas guardadas en los archivos de texto.